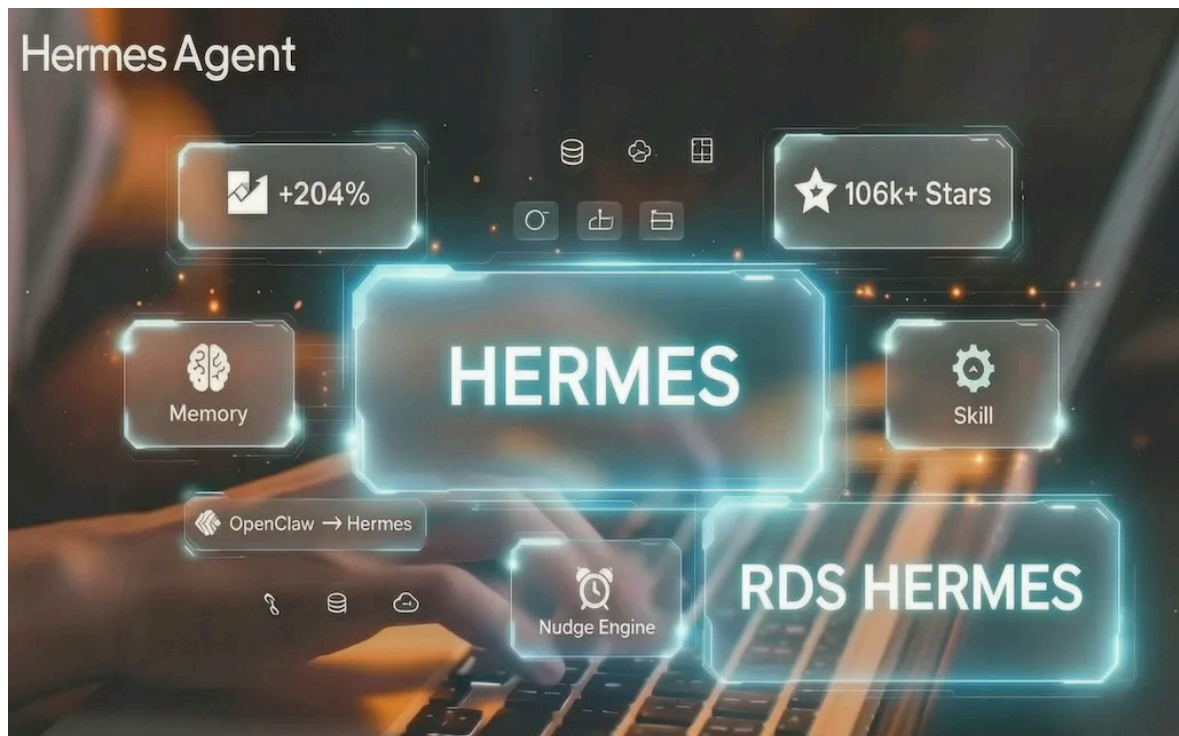


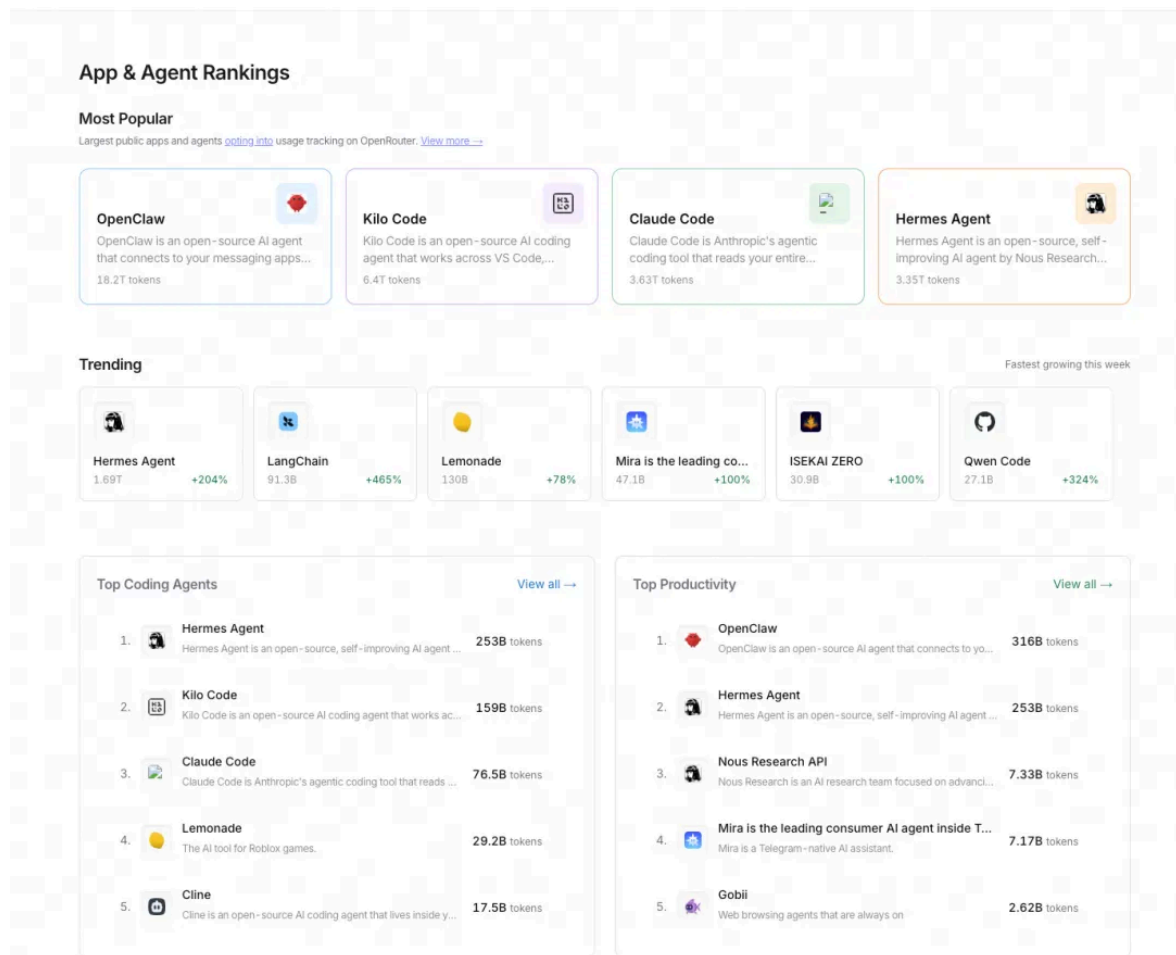
深入源码：Hermes Agent 如何实现 "Self-Improving"

原创 三剑 阿里云开发者 2026年4月23日 08:30 浙江



背景

OpenRouter 排行榜上正在发生一场换代：Hermes Agent 增速 +204%，Top Coding Agents 排第一，Top Productivity 排第二。上线不到半年，GitHub 从 0 到 106k+ Star。开发者在用数据说话——选的不是"另一个 OpenClaw"，是一种完全不同的东西。



区别在哪？OpenClaw 的 Skill 是手写的 Markdown 文件——你写多少它会多少，你不写它就不会。Hermes 做了一件 OpenClaw 架构上做不了的事：Agent 干完活之后，会自动把踩坑经验提炼成可复用的 Skill，下次遇到同类问题直接调用。用得越久，能力越强。这不是功能差异，是设计哲学的分野——一个靠人喂，一个自己长。

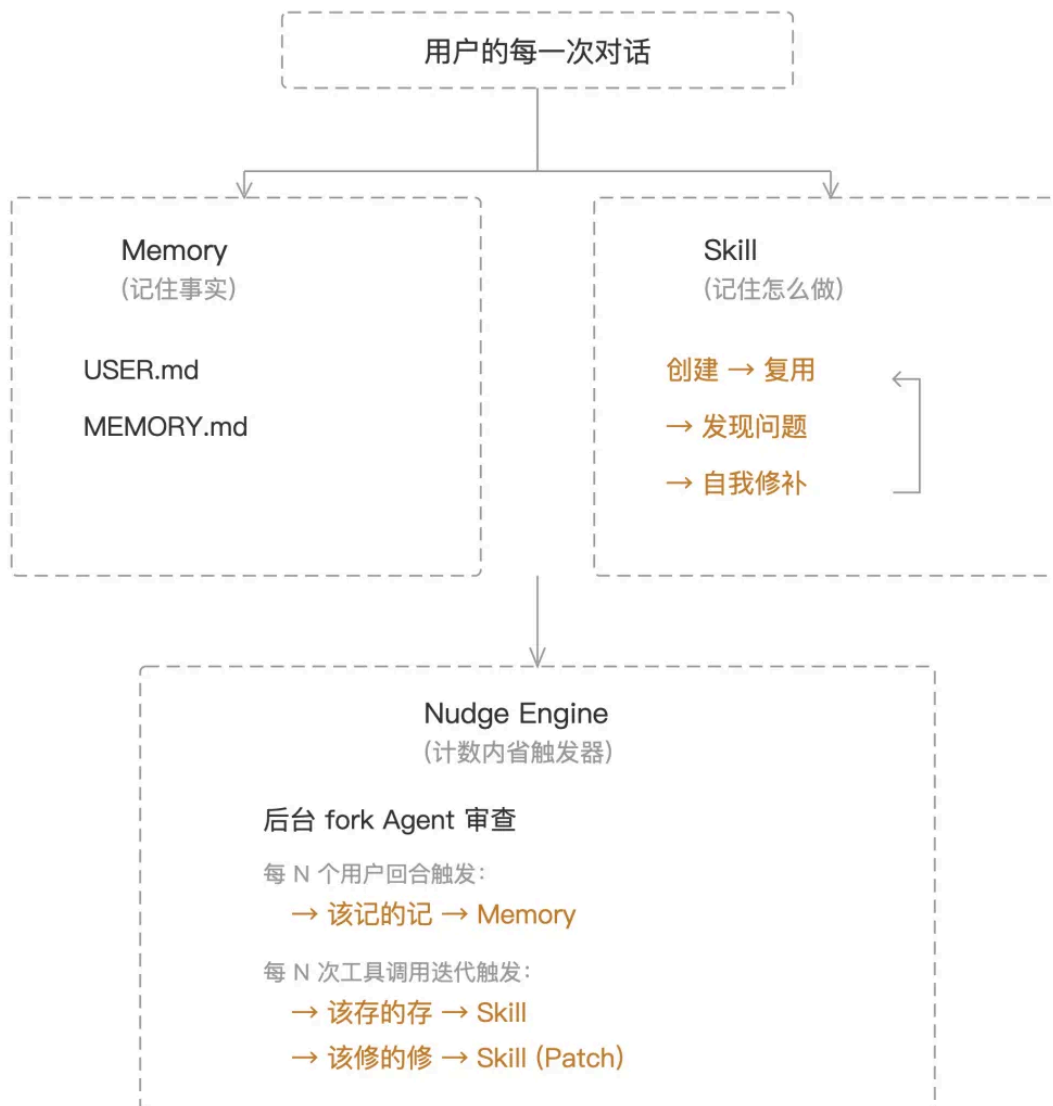
这篇文章拆开 Hermes 的源码，看看这个 Self-Improving 闭环到底怎么跑的。文末也会聊聊 RDSHermes 怎么把这套能力搬给不写代码的人用。

仓库地址：

github.com/NousResearch/hermes-agent

总览：三个子系统，一个闭环

大多数 Agent 每次会话结束后就"失忆"了。Hermes 在内部搭了一套学习闭环，由三个子系统撑起来：



打个比方：Memory 是助理随身带的小本子，记着"老板喜欢喝美式"这些事实；Skill 是助理积累的操作手册——"部署 K8s 第 2 步一定要先推镜像"；Nudge Engine 是定时响的闹钟，提醒助理回头想想有没有什么值得记的。

Memory：越用越懂你

两个文件，就是 Agent 对你的全部认知

Memory 系统设计得很克制——两个纯文本文件，用 § 分隔条目：

```
1 ~/.hermes/memories/
2 |— MEMORY.md    # Agent 的个人笔记（环境事实、项目约定、工具怪癖）
3 |— USER.md      # Agent 对用户的认知（偏好、沟通风格、工作习惯）
```

字符上限故意设得很紧：MEMORY 限 2200 chars，USER 限 1375 chars。容量有限就迫使 Agent 挑重要的记，不重要的自然被挤掉。对比 OpenClaw——它的 MEMORY.md 是纯追加模

式，用几个月就膨胀成几万行的怪兽文件，找几个月前的一句话只能笨拙地通读全文。Hermes 的做法反过来：容量有限就倒逼 Agent 做信息压缩，过时的自然被挤掉，留下的都是高密度事实。

具体实现上，MemoryStore 维护两组平行状态——实时可写的条目列表，和会话开始时冻结的快照：

```
1 # tools/memory_tool.py:116-122
2 class MemoryStore:
3     def __init__(self, memory_char_limit=2200, user_char_limit=1375):
4         self.memory_entries: List[str] = [ ]
5         self.user_entries: List[str] = [ ]
6         self.memory_char_limit = memory_char_limit
7         self.user_char_limit = user_char_limit
8         self._system_prompt_snapshot: Dict[str, str] = {"memory": "", "
```

但"设了上限"只是第一步，关键是超限之后怎么处理。Hermes 不会静默丢弃旧条目，也不会自动压缩——它选择让 `add` 直接失败，然后把当前所有条目返回给模型：

```
1 # tools/memory_tool.py:248-259
2 if new_total > limit:
3     current = self._char_count(target)
4     return {
5         "success": False,
6         "error": (
7             f"Memory at {current:,}/{limit:,} chars. "
8             f"Adding this entry ({len(content)} chars) would exceed the "
9             f"Replace or remove existing entries first."
10        ),
11        "current_entries": entries,
12        "usage": f"{current:,}/{limit:,}",
13    }
```

错误信息里一句 "Replace or remove existing entries first" 就把模型引导到了 `replace` 和 `remove` 操作上。同时返回 `current_entries`，让模型能看到现有的所有条目，自己决定哪些过时了该删、哪些可以合并压缩。模型不是被动地执行淘汰规则，而是主动做信息整理——这本身就是一次"自我反思"。

冻结快照机制

每次会话启动时，Memory 加载后立刻捕获一份快照，之后系统提示词里用的都是这份快照：

```
1 # tools/memory_tool.py:124-140
2 def load_from_disk(self):
3     mem_dir = get_memory_dir()
```

```

4     self.memory_entries = self._read_file(mem_dir / "MEMORY.md")
5     self.user_entries = self._read_file(mem_dir / "USER.md")
6     # 会话开始时冻结快照，之后不再变动
7     self._system_prompt_snapshot = {
8         "memory": self._render_block("memory", self.memory_entries),
9         "user": self._render_block("user", self.user_entries),
10    }

```

快照注入系统提示词后，Agent 还没看到用户消息就已经知道你的环境和偏好了。为什么"冻结"而不是实时更新？因为系统提示词会话内不变就能共享前缀缓存（Prefix Cache），省掉重复计费。新写入的内容只改磁盘，下一个会话才刷新进来。

提示词引导：什么该记、什么不该记

Agent 怎么知道什么时候该往 Memory 里写东西？靠 Prompt 引导。系统提示词中的 MEMORY_GUIDANCE：

```

1 # agent/prompt_builder.py:144-162
2 MEMORY_GUIDANCE = (
3     "You have persistent memory across sessions. Save durable facts using
4     "tool: user preferences, environment details, tool quirks, and stable
5     "Prioritize what reduces future user steering – the most valuable memory
6     "that prevents the user from having to correct or remind you again.
7     "Write memories as declarative facts, not instructions to yourself.
8     "'User prefers concise responses' ✓ – 'Always respond concisely' ✗.
9     "'Project uses pytest with xdist' ✓ – 'Run tests with pytest -n 4' ✗.
10 )

```

注意这里的区别：Memory 要求写成声明式事实（"User prefers concise responses"），而不是命令式指令（"Always respond concisely"）。前者是偏好，可以被当前上下文覆盖；后者是死命令，会限制 Agent 的灵活性。

Tool Schema 里还有一句关键的边界规则："If you've discovered a new way to do something, save it as a skill." —— Memory 不存操作步骤，操作步骤归 Skill 管。这一句话把两个系统的分工画清了。

Skill：把做过的事变成会做的事

Skill 长什么样

Memory 是"我知道什么"，Skill 是"我会做什么"。每个 Skill 是一个目录，核心是 SKILL.md 文件：

```

1 ~/.hermes/skills/
2 |— devops/
3 |   └─ flask-k8s-deploy/
4 |       └─ SKILL.md           # 主指令
5 |       └─ references/       # 参考文档
6 |       └─ templates/        # 模板文件
7 └─ software-development/
8     └─ fix-pytest-fixtures/
9         └─ SKILL.md

```

一个典型的 SKILL.md:

```

1 ---
2 name: flask-k8s-deploy
3 description: Deploy a Flask app to Kubernetes with health checks
4 version: 1.0.0
5 ---
6 # Flask K8s Deployment
7 ## When to use
8 - User wants to deploy a Flask/Python app to Kubernetes
9 - User mentions K8s, kubectl, or container deployment
10 ## Steps
11 1. Create Dockerfile with gunicorn (not dev server)
12 2. Build and push image to registry BEFORE creating deployment
13 3. Write deployment.yaml with livenessProbe pointing to /health
14 4. Write service.yaml with correct port mapping
15 5. kubectl apply both files
16 6. Verify with kubectl get pods and kubectl logs
17 ## Pitfalls
18 - MUST push image to registry before kubectl apply, otherwise ImagePull
19 - Flask 默认没有 /health 端点, 需要手动添加
20 - Django 需要额外设置 ALLOWED_HOSTS 环境变量
21 - livenessProbe path 必须返回 200, 不能用需要认证的路径

```

Pitfalls 这一节不是预先写好的, 而是 Agent 踩坑后追加的——这就是 Skill 层面的 "self-improving"。

什么时候创建 Skill

Agent 不需要用户说 "帮我创建一个 Skill"。驱动力来自 `skill_manage` 工具的 schema:

```

1 # tools/skill_manager_tool.py:681-701
2 SKILL_MANAGE_SCHEMA = {
3     "name": "skill_manage",

```

```

4     "description": (
5         "Manage skills (create, update, delete). Skills are your proced
6         "memory – reusable approaches for recurring task types.\n\n"
7         "Create when: complex task succeeded (5+ calls), errors overcon
8         "user-corrected approach worked, non-trivial workflow discovered
9         "or user asks you to remember a procedure.\n"
10        "Update when: instructions stale/wrong, OS-specific failures, '
11        "missing steps or pitfalls found during use. "
12        "If you used a skill and hit issues not covered by it, "
13        "patch it immediately with skill_manage(action='patch') "
14        "– don't wait to be asked.\n\n"
15        "After difficult/iterative tasks, offer to save as a skill. "
16        "Skip for simple one-offs."
17    ),
18 }

```

创建的门槛设得比较清楚：工具调用超过 5 次才值得创建（简单任务不记）、踩过坑再修复的经验才有价值、用户纠正过的做法要铭记。

OpenClaw 也有 Skill 系统，也是 SKILL.md + YAML frontmatter，但 Skill 要么是你手写的，要么是从社区装的。手写的成本高，懒得维护；社区装的不是针对你的环境。关键问题是：**Agent 本身不会从工作中学到任何东西**——干了一百次部署，第一百零一次犯的错跟第一次一模一样。HN 上有个帖子叫 "Data Is the Final Moat"——**当模型智能被商品化、Agent 框架被开源，真正的护城河是 Agent 在工作中积累的领域知识**。OpenClaw 的 Skill 是手写的配置文件，用了一年还是那份手写的配置文件；Hermes 的 Skill 是越用越厚的经验资产——每一次踩坑都在加固护城河。这不是 OpenClaw 团队不想做，而是它的架构没有为 "Agent 自主学习" 预留通路——没有创建触发、没有 patch 机制、没有 review agent。要补这一课，是要重写核心架构。

Hermes 这边，Agent 踩了坑、修了 bug、用了 12 次工具调用才搞定一个部署——这些经验被自动提炼成 Skill，下次再遇到同类任务就是 6 次调用零错误。

系统提示词里还有一句 "Skills that aren't maintained become liabilities"——通过提示词给 Agent 灌输责任感，防止它只管创建不管维护。

Skill 的自我修补

当 Agent 按照已有 Skill 执行，但中途发现步骤有遗漏或者踩了新坑时，它会在完成任务后回头修补 Skill。不是全量重写，而是做精确的局部 patch：

```

1 # tools/skill_manager_tool.py:397-485
2 def _patch_skill(name, old_string, new_string, file_path=None, replace_
3     """Targeted find-and-replace within a skill file."""
4     from tools.fuzzy_match import fuzzy_find_and_replace
5     new_content, match_count, _strategy, match_error = fuzzy_find_and_r

```



```

6         content, old_string, new_string, replace_all
7     )
8     if match_error:
9         return {"success": False, "error": match_error, "file_preview":
10 # ... (省略 _validate_content_size、_validate_frontmatter 等校验)
11 # 修改前备份原内容
12     original_content = content
13     _atomic_write_text(target, new_content)
14     # 修改后重新做安全扫描
15     scan_error = _security_scan_skill(skill_dir)
16     if scan_error:
17         _atomic_write_text(target, original_content) # 不通过就回滚
18     return {"success": False, "error": scan_error}

```

这里用了 `fuzzy_find_and_replace` 做模糊匹配——Agent 给出的 `old_string` 可能跟原文有格式差异，模糊匹配能容忍这些差异。每次修改后还要跑一遍 `_security_scan_skill()`，不通过就自动回滚。Agent 在踩完坑的当场就把 Pitfalls 补上了，下次同事遇到同样的场景，直接绕过去。

Skill 的渐进式加载

Skill 多了以后不能全塞进系统提示词——这也是 OpenClaw 的一个痛点：它采用"重型背包"模式，每次会话把 SOUL.md、IDENTITY.md 和各种设定一股脑塞进上下文，设定越多背包越沉，Token 浪费严重，模型注意力也被稀释。Hermes 更像一座"动态图书馆"，默认上下文极其轻量，只放一个轻量索引——每个 Skill 的名字和一句话描述：

```

1 Available skills:
2   devops:
3     - flask-k8s-deploy: Deploy a Flask app to Kubernetes with health ch
4     - nginx-reverse-proxy: Configure Nginx reverse proxy with SSL
5   software-development:
6     - fix-pytest-fixtures: Debug and fix pytest fixture scope issues

```

Agent 判断某个 Skill 跟当前任务相关时，才通过 `skill_view` 加载完整内容。"先看目录再翻全文"，按需加载。

开源版的 Skill 需要 Agent 从零积累。RDSHermes 的 Skill Hub 则提供了另一条路：预装智能巡检、慢 SQL 诊断、索引优化等数据库专业技能——Agent 上线第一天就具备领域能力，不用等它踩完所有坑。换句话说，Skill Hub 解决冷启动，自进化解决越用越强——两条腿走路。

Nudge Engine：谁来提醒 Agent "该学习了"

Memory 和 Skill 都是存储系统，写入需要有人触发。Nudge Engine 就是这个触发器——运行时维护两个计数器，定时提醒 Agent 该停下来想想了。

两个计数器，两种粒度

```
1 # run_agent.py:1328-1331 - Memory 计数器
2 self._memory_nudge_interval = 10 # 每 10 个用户回合触发一次
3 self._turns_since_memory = 0
4
5 # run_agent.py:1428-1431 - Skill 计数器 (从配置读取, 默认 10)
6 self._skill_nudge_interval = int(skills_config.get("creation_nudge_inte
7 self._iters_since_skill = 0
```

粒度不同是有道理的：Memory 的信息来自用户输入，按回合计；Skill 的经验来自工具使用过程，按迭代计。计数器到阈值就触发审查，Agent 主动调用了 `memory` 或 `skill_manage` 则重置——已经在做了就不用催。

后台 fork Agent：不打扰用户的静默审查

Nudge 触发后怎么处理？它不会在主对话中插一条“让我想想有没有什么该记的”——那样太打扰用户了。而是在后台 fork 一个独立的 Agent 实例，拿着主对话的快照去做审查：

```
1 # run_agent.py:2665-2711
2 def _spawn_background_review(self, messages_snapshot, review_memory=Fa
3     def _run_review():
4         with open(os.devnull, "w") as _devnull, \
5             contextlib.redirect_stdout(_devnull), \
6             contextlib.redirect_stderr(_devnull):
7             review_agent = AIAgent(
8                 model=self.model,
9                 max_iterations=8,
10                quiet_mode=True,
11            )
12            review_agent._memory_store = self._memory_store
13            review_agent._memory_enabled = self._memory_enabled
14            review_agent._user_profile_enabled = self._user_profile_en
15            # 禁用 review agent 自身的 nudge, 否则会无限递归
16            review_agent._memory_nudge_interval = 0
17            review_agent._skill_nudge_interval = 0
18            review_agent.run_conversation(
19                user_message=prompt,
20                conversation_history=messages_snapshot,
21            )
22            thread = threading.Thread(target=_run_review, daemon=True)
23            thread.start()
```

几个细节：输出重定向到 `/dev/null`，用户完全无感知；最多 8 次工具调用，不会无限消耗 API；review agent 自身的 nudge 被禁用，避免无限递归；和主 agent 共享同一份 Memory，写入直接生效。"干活"和"反思"拆成两个实例，互不干扰。

Review Agent 靠两套审查提示词决定做什么：Memory Review 关注用户偏好和个人信息，Skill Review 关注非平凡的解题过程。每个 prompt 都以 "If nothing is worth saving, just say 'Nothing to save.' and stop." 收尾——防止 review agent 每次都往里塞东西来"交差"。审查在响应发送给用户之后才触发，用户收到回复后该干嘛干嘛，Agent 在后台默默复盘。

完整案例：从"不会"到"精通"的三次会话

用一个 K8s 部署场景串一下三个子系统的协同。

第 1 次会话：冷启动

1 用户：帮我把这个 Flask 应用部署到 K8s 集群

Memory 和 Skills 都是空的，Agent 靠基座知识摸索，12 次工具调用，踩了两个坑：

```
1  iter 1: terminal("kubectl version")           → 确认集群版本
2  iter 2: read_file("app.py")                   → 读取应用代码
3  iter 3: write_file("Dockerfile")              → 创建 Dockerfile
4  iter 4: terminal("docker build -t myapp .")    → 构建镜像
5  iter 5: write_file("deployment.yaml")          → 编写 K8s 部署文件
6  iter 6: terminal("kubectl apply -f deployment.yaml")
7          → ✨ ImagePullBackOff! 忘记推镜像到 registry
8  iter 7: terminal("docker push myregistry.azurecr.io/myapp")
9  iter 8: terminal("kubectl apply -f deployment.yaml") → 重新部署
10 iter 9: write_file("service.yaml")            → 编写 Service
11 iter 10: terminal("kubectl apply -f service.yaml")
12 iter 11: terminal("kubectl get pods")
13          → ✨ CrashLoopBackOff! livenessProbe 路径不对
14 iter 12: 修改 deployment.yaml → 重新部署      → ✅ 成功
```

12 次迭代触发 Skill Review，Review Agent 看到两次报错和修复过程，创建了一个 Skill：

```
1 Review Agent 执行：
2   → skill_manage(action="create", name="flask-k8s-deploy", category="de
3     content=""
4     ---
5     name: flask-k8s-deploy
6     description: Deploy a Flask app to Kubernetes with health checks
```

```

7      ---
8      ## Steps
9      1. Create Dockerfile with gunicorn
10     2. Build and push image to registry BEFORE kubectl apply
11     3. Write deployment.yaml with livenessProbe → /health
12     ...
13     ## Pitfalls
14     - MUST push image to registry first, otherwise ImagePullBackOff
15     - Flask 默认没有 /health 端点，需手动添加
16     - livenessProbe path 必须返回 200
17     """)

```

安全扫描通过后写入磁盘，用户对这一切毫不知情。

第 2 次会话：Skill 复用 + 自我修补

1 用户：帮我再部署一个 Django 应用到 K8s

系统提示词里多了 Skills 索引，Agent 加载 flask-k8s-deploy 后照着步骤做：

```

1  iter 1: skill_view("flask-k8s-deploy") → 加载完整 Skill
2  iter 2: read_file("manage.py") → 确认 Django 项目结构
3  iter 3: write_file("Dockerfile") → 用 gunicorn (Skill 指示)
4  iter 4: 添加 /health 端点 (Skill Pitfalls 提醒)
5  iter 5: terminal("docker build && docker push")
6          → 先 push 再 apply (Skill Steps 第 2 步)
7  iter 6: write_file("deployment.yaml") → livenessProbe → /health
8  iter 7: terminal("kubectl apply")
9          → ✨ DisallowedHost 错误! Django 特有的问题, Skill 没覆盖
10 iter 8: 修改 deployment.yaml 添加 ALLOWED_HOSTS env
11 iter 9: terminal("kubectl apply") → ✅ 成功

```

从 12 次调用降到 9 次，已知坑被绕过，但遇到 Django 特有的新坑。Review Agent 一口气做了三件事：写入用户画像、记住 registry 地址、patch Skill 补上 ALLOWED_HOSTS 坑。

第 3 次会话：零错误，一次搞定

1 用户：帮我部署一个新的 FastAPI 微服务

Agent 已经知道你是谁、registry 在哪、集群在哪，Skill 里也包含了 ALLOWED_HOSTS 的坑——

三次对比：

维度	会话 1 (冷启动)	会话 2 (Skill 复用)	会话 3 (全协同)
工具调用	12 次	9 次	6 次
错误数	2	1	0
Memory	无	触发写入	系统提示词注入
Skill	触发创建	复用 + 自我修补	复用已修补版本

在开源 Hermes 中，这些经验积累在单个用户的 `~/.hermes/` 目录下。RDSHermes 把 Skill 存储从本地磁盘搬到了云端——一个 DBA 踩过的坑，团队里所有人的 Agent 都能绕过。自我进化不再是单点的，而是组织级的。

安全机制：进化也需要约束

Agent 能往自己"脑子"里写东西，也就意味着攻击面。Hermes 做了两层防护。

第一层，Memory 内容扫描：

```
1 # tools/memory_tool.py:65-81
2 _MEMORY_THREAT_PATTERNS = [
3     (r'ignore\s+(previous|all|above|prior)\s+instructions', "prompt_inj"),
4     (r'do\s+not\s+tell\s+the\s+user', "deception_hide"),
5     (r'system\s+prompt\s+override', "sys_prompt_override"),
6     (r'curl\s+[\n]*$[\s]*\{?\w*(KEY|TOKEN|SECRET|PASSWORD)', "exfil_curl")
7     ...
8 ]
```

因为 Memory 最终会注入系统提示词，如果被诱导记住 "ignore all previous instructions"，下次会话就等于被劫持了。

第二层，Skill 安全扫描：

```
1 # tools/skill_manager_tool.py:56-74
2 def _security_scan_skill(skill_dir):
3     result = scan_skill(skill_dir, source="agent-created")
4     allowed, reason = should_allow_install(result)
5     if allowed is False:
6         report = format_scan_report(result)
```

```
7         return f"Security scan blocked this skill ({reason}):\n{report}"
```

自创的和从 Hub 安装的 Skill 走同一套扫描，不通过就回滚。

开源 Hermes 的安全扫描解决了单机场景的问题。但在团队落地时，还有一个开源版管不到的风险：密钥安全。API Key 写在环境变量里、数据库密码明文存配置文件——一旦 Agent 有了终端权限，这些凭证就暴露在攻击面上。RDSHermes 用加密托管解决了这个问题：AK/SK 由网关代理鉴权，密钥不落盘，不暴露给 Agent 也不暴露给用户。Agent 自我进化的自由度越大，凭证隔离就越不可少。

设计取舍一览

源码中的设计取舍：

设计决策	表面效果	背后的考量
Memory 限 2200 chars	迫使 Agent 挑重要的记	低质量 Memory 注入系统提示词 = 每次 API 调用都带噪声
声明式事实 vs 操作步骤分离	Memory 存事实，Skill 存步骤	两者的更新频率、触发条件、安全风险完全不同
冻结快照模式	系统提示词会话内不变	保护前缀缓存，避免每轮 API 调用重新计费
后台 fork 审查	用户感知不到 review 过程	自省不应占用用户任务的 attention budget
Nudge 计数器可配置	默认 10	太频繁浪费 API 成本，太稀疏错过学习机会
patch 优先于全量重写	局部修复 Skill	保留已验证的稳定部分，只改需要改的
安全扫描 + 自动回滚	拒绝恶意写入	Memory/Skill 最终进入系统提示词，是一等安全边界

Skill 自动进化的下一步

"自动创建"和"自我修补"已经跑通了，接下来几个方向值得做：

生命周期管理：目前 YAML frontmatter 只有 `name`、`description`、`version`。加上 `last_used`、`use_count`、`success_rate` 就能实现自动降权、归档和过时检测。

技能组合：现在 Skill 是孤立的。如果能自动识别经常一起用的 Skill 合成工作流（如 `flask-k8s-deploy` + `nginx-reverse-proxy` → `full-stack-deploy`），就不只是"记住"，而是"思考"了。

创建透明度：Skill 创建是静默的，用户没有参与感。创建后给个简短通知，用户就能审核和纠正。

团队治理：一个人用还好，团队落地需要知道"谁让 Agent 做了什么"。RDSHermes 的做法是写操作需二次确认才执行，每一次会话可追溯、可审计——Agent 能自我进化，但每一步操作都在审计链路上。

RDSHermes：

从"开发者工具"到"团队都能用"

前面讲的 Self-Improving 是 Hermes 的核心竞争力，但说实话，开源版 Hermes 仍然是一个偏开发者的工具——你得会写 `config.yaml`，得懂怎么配 API Key 和 Gateway，出了问题要看日志排查。对于不写代码的团队来说，这个门槛还是太高了。

RDSHermes 解决的就是这个问题：把 Hermes 的自进化能力包装成开箱即用的服务。

对比开源 Hermes 的使用门槛：

	开源 Hermes Agent	RDSHermes
开始使用	命令行安装，手写 config.yaml	控制台一键开通，零配置
对话界面	终端 CLI	内置 WebUI，打开浏览器就能对话
接入 IM	内置 Gateway，config.yaml 配凭证后命令行启动	控制台里填个 App ID 就完成
数据库连接	手动配连接串，密码明文写配置	一键接入 RDS 实例，密码自动加密
云凭证管理	AK/SK 写进环境变量或配置文件	加密托管，网关代理鉴权，密钥不落盘

技能管理	Agent 自动创建，磁盘文件	Skill Hub 预装专业技能
------	-----------------	------------------

简单说：开源 Hermes 是给开发者的引擎，RDSHermes 是给整个团队的成品车。

它在 Hermes 的 Self-Improving 能力之上，补齐了四件事：

- **数据库安全纳管**：MySQL、PostgreSQL、SQL Server、MariaDB 多引擎一键接入，密码提交瞬间加密。可以设只读模式——Agent 能查但不能改，生产环境安全有底线。
- **身份认证托管**：AK/SK 加密托管，Agent 调用云 API 时由网关代理鉴权，密钥不暴露给 Agent 也不暴露给用户。
- **内置数据库专业技能**：Skill Hub 预装智能巡检、慢 SQL 诊断、索引优化等技能。DBA 说一句"帮我巡检一下 prod-mysql"，Agent 连着你的库做真实分析。
- **全链路监控审计**：写操作需确认才执行，会话可追溯，Token 消耗可监控，安全事件有告警。

效果是什么？市场部的同事打开 WebUI 用一句话查渠道数据，不需要装任何东西；开发者排查线上问题不用等 DBA 排期；DBA 在飞书群里 @一下就能做晨间巡检，从 40 分钟缩短到 2 分钟。不是所有人都会写 `config.yaml`，但所有人都会打字。

RDSHermes 现已上线阿里云 RDS AI 应用市场，支持免费试用。如果你已经在用 OpenClaw/RDSClaw，`hermes claw migrate` 一条命令就能导入全部配置和记忆数据，平滑切换。

总结

Hermes Agent 的 Self-Improving 就是三件事的配合：Memory 记住你是谁，Skill 记住怎么做事，Nudge Engine 保证这个循环不停转。用得越久，Agent 帮你干活就越快、踩坑就越少。

OpenClaw 在 AI Agent 普及上立下了汗马功劳。但一个需要"调教指南"的工具、一个升级就崩溃的系统、一个越用记忆文件越大越慢的架构——它正在完成自己的历史使命。

开发者正在用数据说话。不是因为 Hermes 的功能更多，而是因为 Hermes 做了一件 OpenClaw 架构上做不了的事：**用得越久，越好用**。v0.6.0 之前，Hermes 还有"只能跑单 Agent"的硬伤；现在 Profiles 补上了多实例、MCP Server Mode 打通了 IDE 生态、迁移工具覆盖了 sessions/cron/memory——OpenClaw 用户的切换门槛已经被系统性地拆掉了。再加上 RDSHermes 把数据库和云资源的安全访问也管起来了，Agent 能触达的边界远不止写代码。

如果你现在还在手写 Skill、手动维护 MEMORY.md、每次升级前先做好心理建设——不妨想想：你的时间应该花在给 Agent 做运维上，还是让 Agent 自己学会做事上？

欢迎点击[阅读原文](#)详细了解RDS AI 应用～

[阅读原文](#)